

Homework — Signal detection (SignalDetection class)

Purpose

Implement a **signal detection theory (SDT)** class in Python, then extend it with operator overloading and plotting.

Background (formulas)

Use **hits**, **misses**, **false alarms**, and **correct rejections** as counts from a yes/no detection experiment.

- **Hit rate**
 $H = \text{hits} / (\text{hits} + \text{misses})$ (avoid dividing by zero if both are zero; same for FA rate below.)
- **False alarm rate**
 $FA = \text{false_alarms} / (\text{false_alarms} + \text{correct_rejections})$
- Let Z be the **standard normal quantile** (inverse CDF), `scipy.stats.norm.ppf` in Python.
- **Sensitivity:**
 $d' = Z(H) - Z(FA)$
- **Criterion (C):**
 $C = -0.5 * (Z(H) + Z(FA))$

Background material on SDT

- Introduction to Signal Detection Theory. [video] [slides]
- Signal Detection Theory modeling of recognition memory. [video] [slides]
- An application of signal detection theory to shape perception. [video] [slides]
- Text by David Heeger [pdf]

Requirements

Interface

- ☐ Put your implementation in a file named `signal_detection.py`.
- ☐ Define a class named `SignalDetection`.
- ☐ Use these method names and argument orders exactly:
 - `SignalDetection(hits, misses, false_alarms, correct_rejections)`
 - `hit_rate(self)`
 - `false_alarm_rate(self)`
 - `d_prime(self)`
 - `criterion(self)`
 - `__add__(self, other)`
 - `__sub__(self, other)`
 - `__mul__(self, factor)`
 - `plot_sdt(self)` and/or `plot_roc(sdt_list)` (as a `@staticmethod`)

Core class

- ☐ Constructor `SignalDetection(hits, misses, false_alarms, correct_rejections)` storing the four counts.

- `hit_rate()` and `false_alarm_rate()` returning floats in $[0, 1]$ when defined; document or handle edge cases where denominators are zero (tests use only well-behaved counts).
- `d_prime()` and `criterion()` matching the formulas above using `scipy.stats.norm.ppf`.
- Input validation that prevents corrupt object state (for example: negative counts, non-numeric values, invalid operator inputs).

Make sure the class cannot be corrupted by passing in invalid data or by incorrectly using the class methods.

Operators (instance methods)

- `__add__(self, other)` — new `SignalDetection` whose four counts are **elementwise sums** of two instances.
- `__sub__(self, other)` — new `SignalDetection` whose four counts are **elementwise differences** of two instances.
- `__mul__(self, factor)` — new `SignalDetection` whose four counts are **multiplied by a scalar factor**.

Plotting

Use `matplotlib` to create at least one of these:

- `plot_sdt(self)` — **SDT plot**: overlapping **signal** and **noise** normal distributions (same variance, sensible x-range), a vertical line for the **criterion**, and a visual indication of d' (e.g. horizontal arrow or segment between distribution means). Label axes and include a title or legend so the figure is interpretable.
- `plot_roc(sdt_list)` — **static** method taking a **sequence** of `SignalDetection` instances. Plot an **ROC-style** curve: **hit rate** on the **horizontal** axis, **false alarm rate** on the **vertical** axis, one point per object; include **(0,0)** and **(1,1)** on the curve by definition. Label axes.
- Plot methods should return matplotlib objects (for example `fig`, `ax`).

Reading

Course module: Object-oriented programming · Poldrack — Software engineering (objects, APIs).